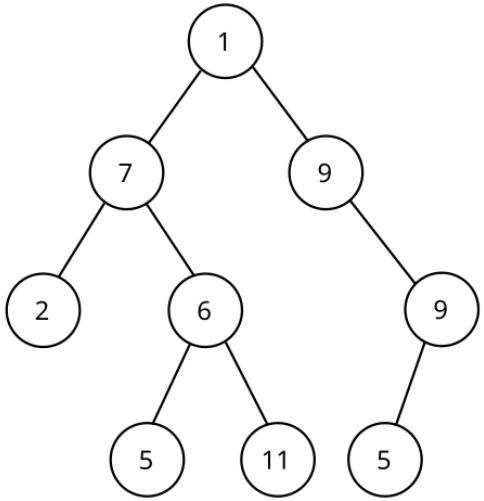
# Lecture 18

## Searching and Sorting I



# Announcements

- Office hours are up on course website
  - T: 12pm-4pm; F: 11am-1pm
  - Schedule an appointment individually or as a group

- Project II deliverable 1 due 11/13

# Last time

- Finished up dynamic programming
- Lab: worked through the integer knapsack problem

# Today

Search algorithms

- Linear and binary search

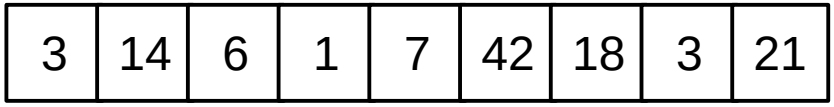
Sorting algorithms

- Selection sort
- Insertion sort

# Problem

Given a list `L`, determine whether or not `L` contains target element `t`

- Either return index containing `t` or return `-1`
- `L` is an arbitrary list



# Linear Search

Exhaustively search through all of the indices (at most once) until

- You find `t`, then return its index
- You've searched all indices, then return `-1`

# Linear Search

Exhaustively search through all of the indices (at most once) until

- You find `t`, then return its index
- You've searched all indices, then return `-1`

Running time: `O(n)`

Can we do better?

# Linear Search

Exhaustively search through all of the indices (at most once) until

- You find `t`, then return its index
- You've searched all indices, then return `-1`

Running time: $O(n)$

Can we do better?

- No: if you look at fewer than n indices, then you may miss one containing the target

# Linear Search

Exhaustively search through all of the indices (at most once) until

- You find `t`, then return its index
- You've searched all indices, then return `-1`

Running time: $O(n)$

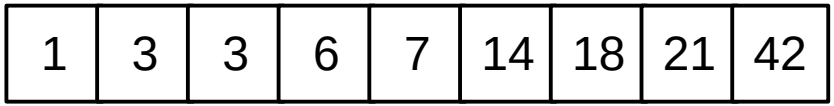
Can we do better?

- No: if you look at fewer than `n` indices, then you may miss one containing the target
- Any search algorithm on an arbitrary list is $\Omega(n)$

# Problem

Given a sorted list `L`, determine whether or not `L` contains target element `t`

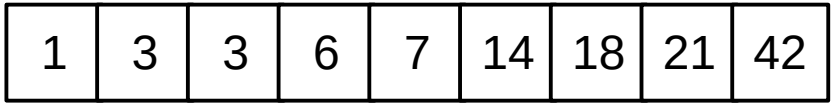
- Either return index containing `t` or return `-1`
- Assumes `L` contains elements that can be ordered



# Problem

Given a sorted list `L`, determine whether or not `L` contains target element `t`

- Either return index containing `t` or return `-1`
- Assumes `L` contains elements that can be ordered
- Can we do better than linear search?



# Binary Search

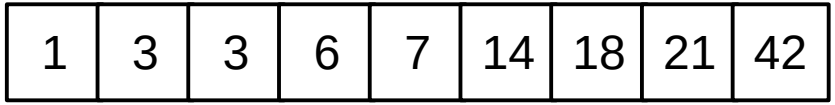
Iterative compare the middle element `m` to the target `t`

- If `m = t`, return the index of `m`
- If `m > t`, run search for elements to the left of `m`
- If `m < t`, run search for elements to the right of `m`
- If you're out of indices to search, then return `-1`

# Binary Search

Iterative compare the middle element `m` to the target `t`

- If `m = t`, return the index of `m`
- If `m > t`, run search for elements to the left of `m`
- If `m < t`, run search for elements to the right of `m`
- If you're out of indices to search, then return `-1`

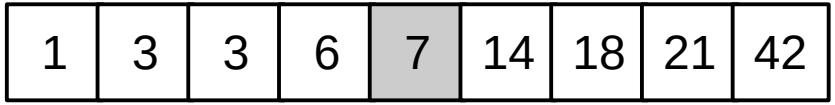



Is 14 in the list?

# Binary Search

Iterative compare the middle element `m` to the target `t`

- If `m = t`, return the index of `m`
- If `m > t`, run search for elements to the left of `m`
- If `m < t`, run search for elements to the right of `m`
- If you're out of indices to search, then return `-1`



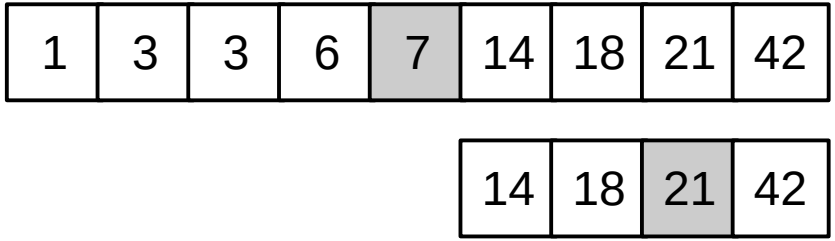

Is 14 in the list?

# Binary Search

Iterative compare the middle element `m` to the target `t`

- If `m = t`, return the index of `m`
- If `m > t`, run search for elements to the left of `m`
- If `m < t`, run search for elements to the right of `m`
- If you're out of indices to search, then return `-1`

Is 14 in the list?

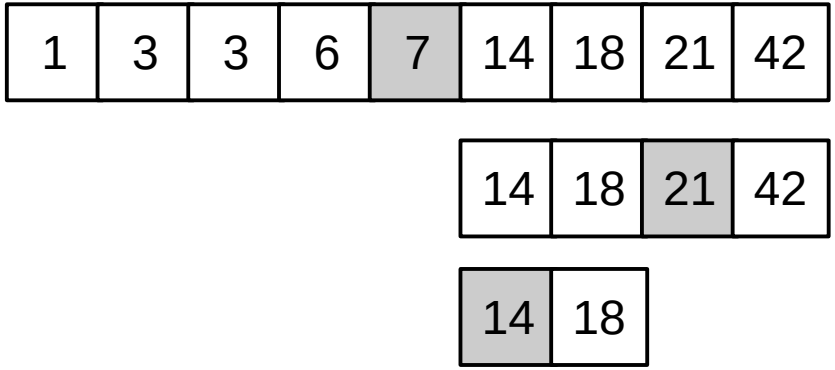


# Binary Search

Iterative compare the middle element `m` to the target `t`

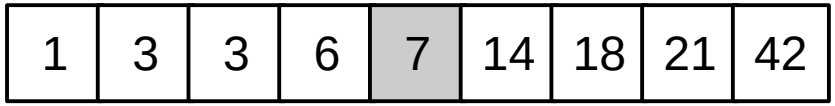
- If `m = t`, return the index of `m`
- If `m > t`, run search for elements to the left of `m`
- If `m < t`, run search for elements to the right of `m`
- If you're out of indices to search, then return `-1`

Is 14 in the list?



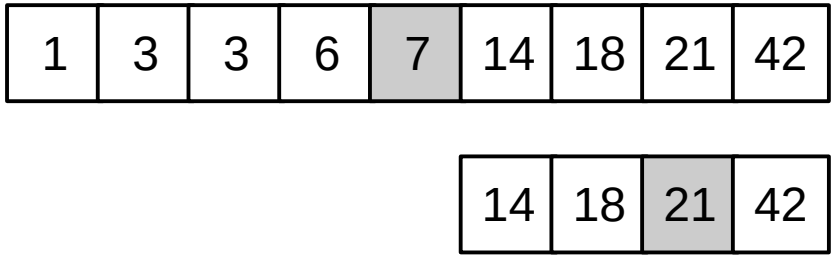
# Binary Tree

- Is 19 in the list?



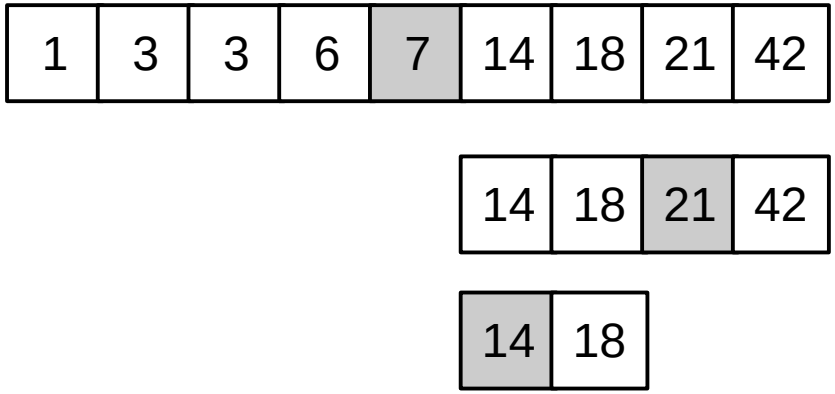
# Binary Tree

- Is 19 in the list?



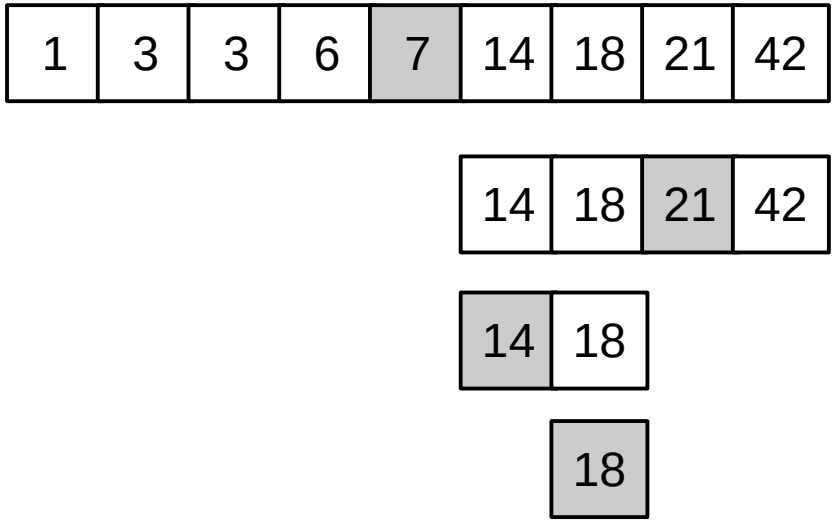
# Binary Tree

- Is 19 in the list?



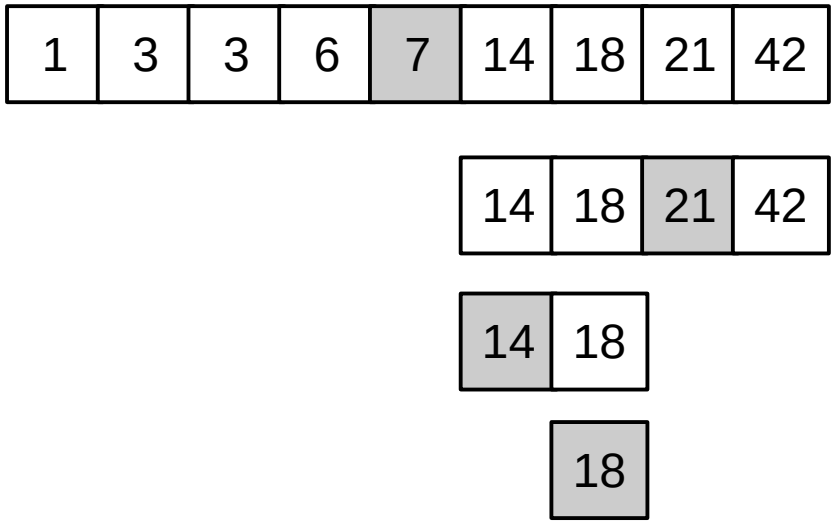
# Binary Tree

- Is 19 in the list?



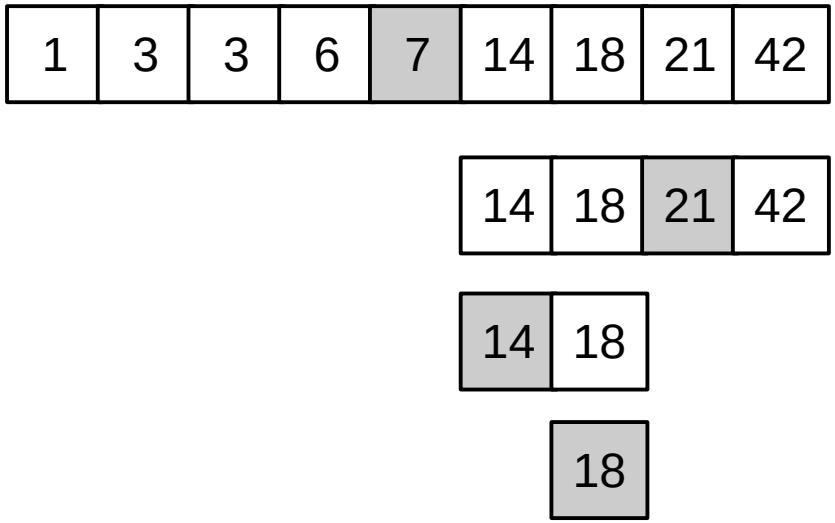
# Binary Tree

- Is 19 in the list? No, return -1

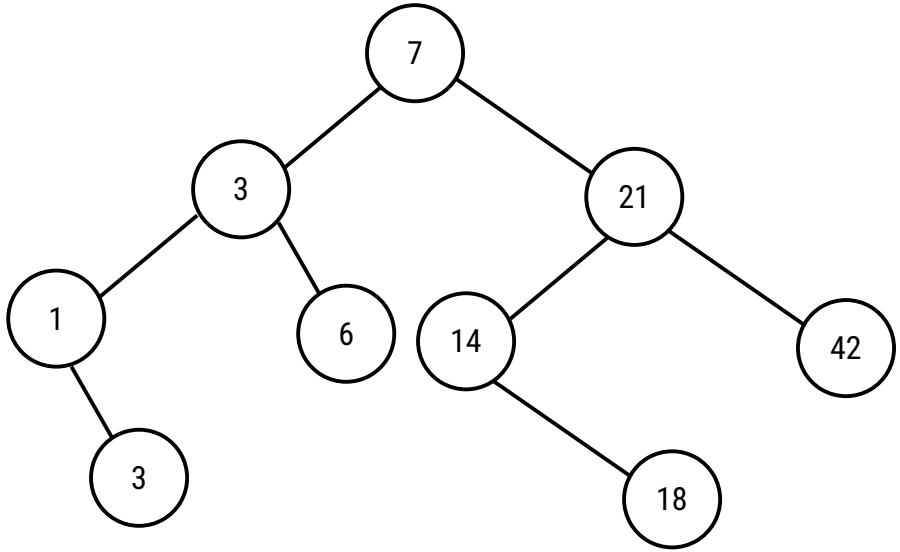


# Binary Tree

- Is 19 in the list? No, return -1

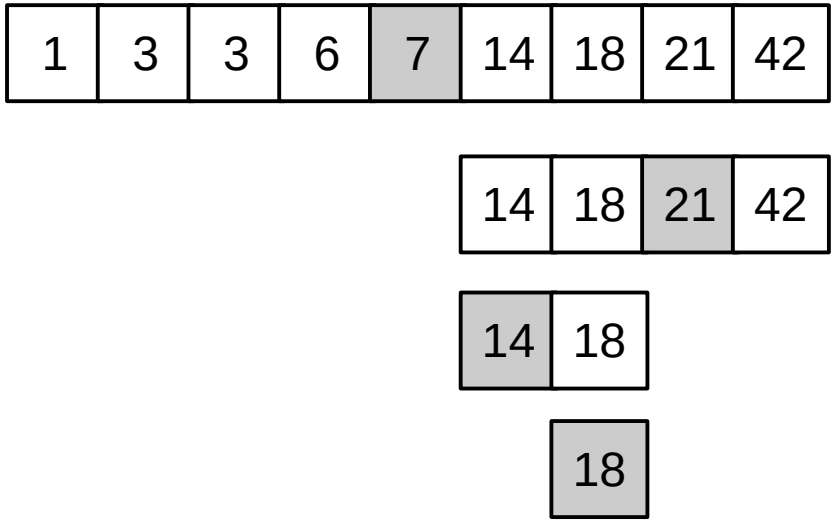



- Represent binary search as a tree

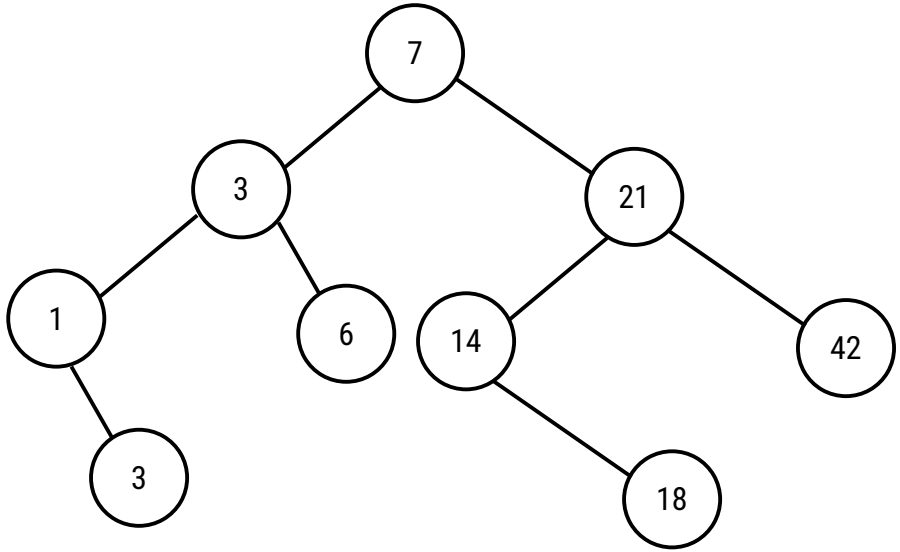


# Binary Tree

- Is 19 in the list? No, return -1



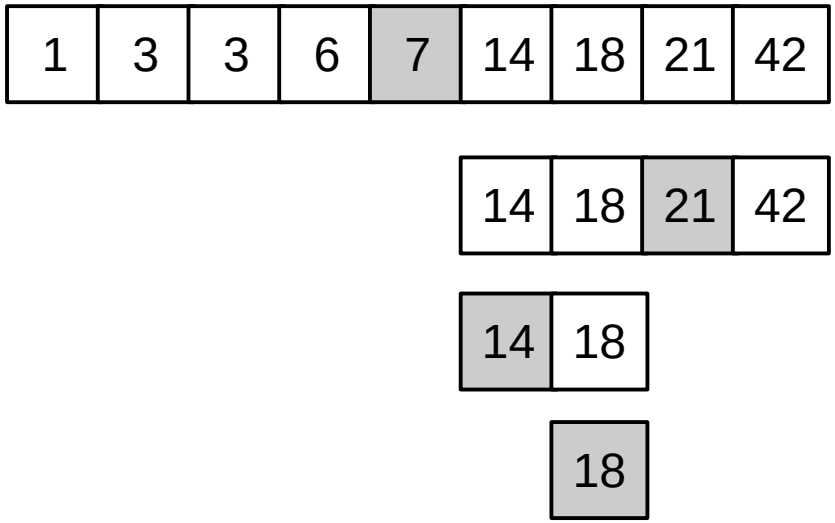

- Represent binary search as a tree



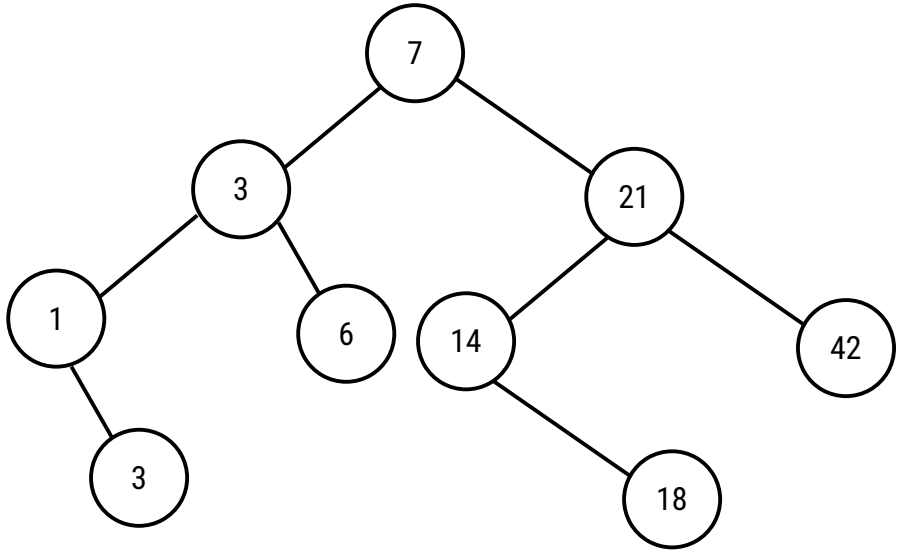

Tree height: 3

# Binary Tree

- Is 19 in the list? No, return -1



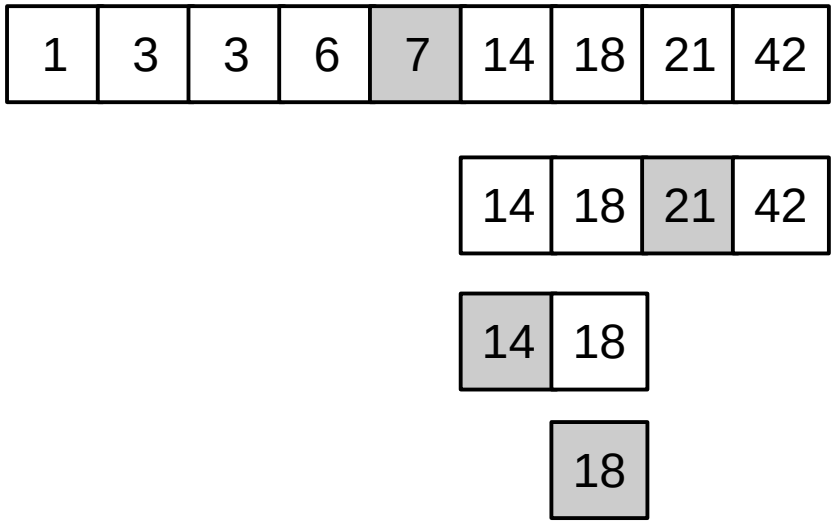

- Represent binary search as a tree



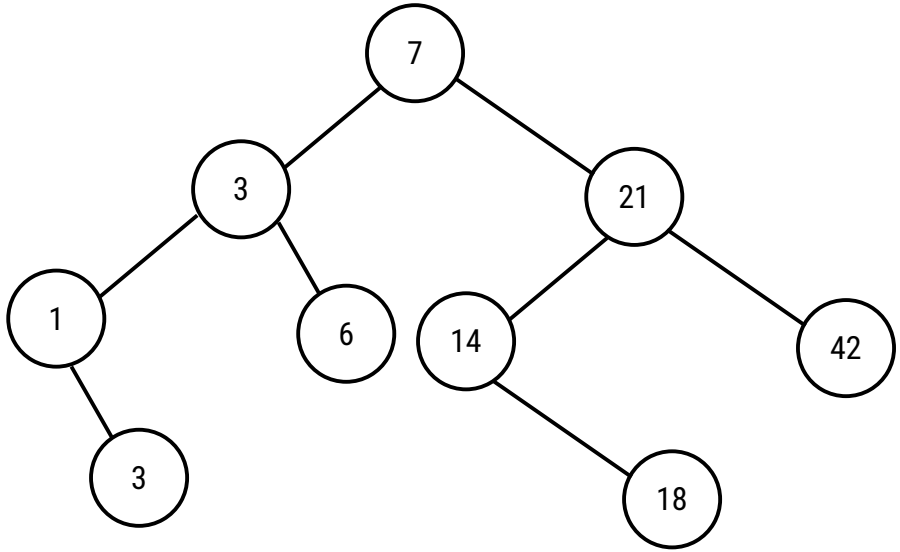

Tree height: $3 = \lceil \log n \rceil \leq \log n + 1$

# Binary Tree

- Is 19 in the list? No, return -1




- Represent binary search as a tree

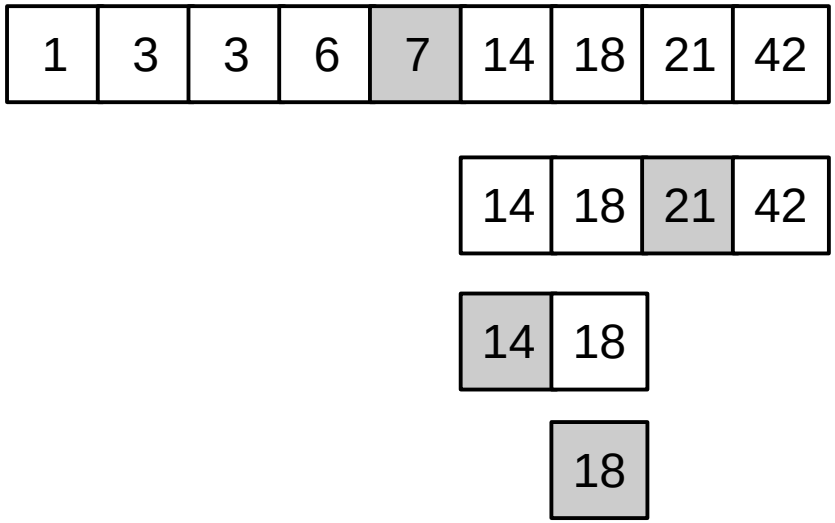



Running time:

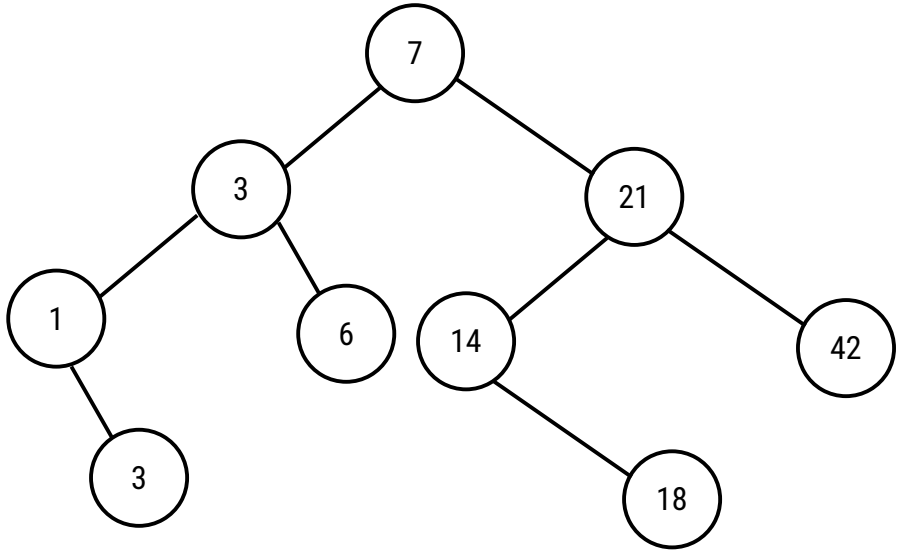
Tree height: $3 = \lceil \log n \rceil \leq \log n + 1$

# Binary Tree

- Is 19 in the list? No, return `-1`




- Represent binary search as a tree




Running time: $\Theta(\log n)$

Tree height: $3 = \lceil \log n \rceil \leq \log n + 1$

# Recap

Searching an arbitrary list: $O(n)$

Searching a sorted list: $O(\log n)$

# Recap

Searching an arbitrary list: $O(n)$

Searching a sorted list: $O(\log n)$

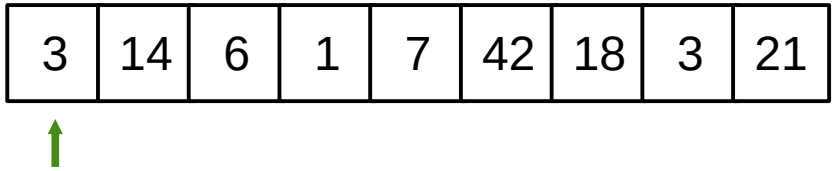
Q: can we beat linear search by sorting first, then running binary search?

- What's the running time of sorting?

# Selection Sort

Initialize a pointer at index 0

- Left of the pointer is sorted




Find min value from pointer

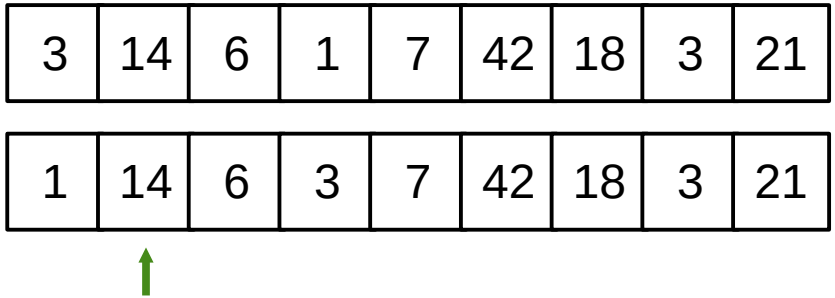
- Swap with pointer value
- Increment pointer

Continue until pointer reaches end of list

# Selection Sort

Initialize a pointer at index 0

- Left of the pointer is sorted




Find min value from pointer

- Swap with pointer value
- Increment pointer

Continue until pointer reaches end of list

# Selection Sort

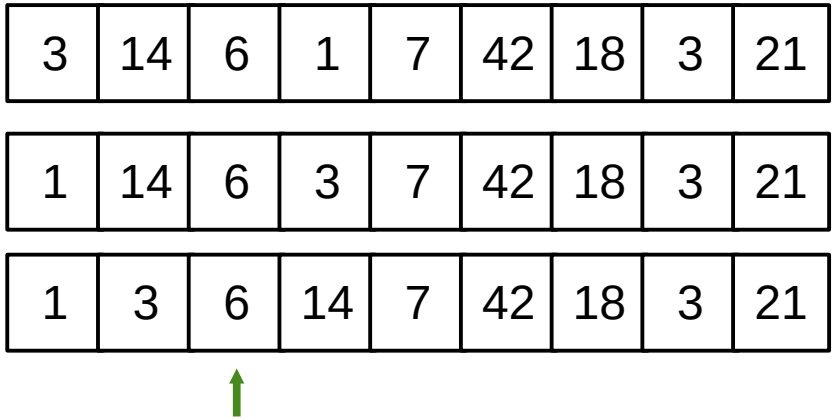
Initialize a pointer at index 0

- Left of the pointer is sorted

Find min value from pointer

- Swap with pointer value
- Increment pointer

Continue until pointer reaches end of list



# Selection Sort

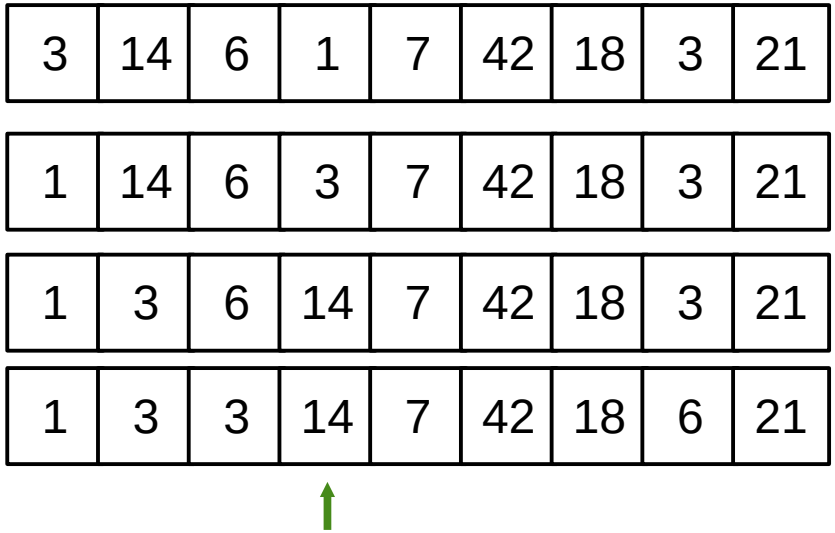
Initialize a pointer at index 0

- Left of the pointer is sorted

Find min value from pointer

- Swap with pointer value
- Increment pointer

Continue until pointer reaches end of list



# Selection Sort

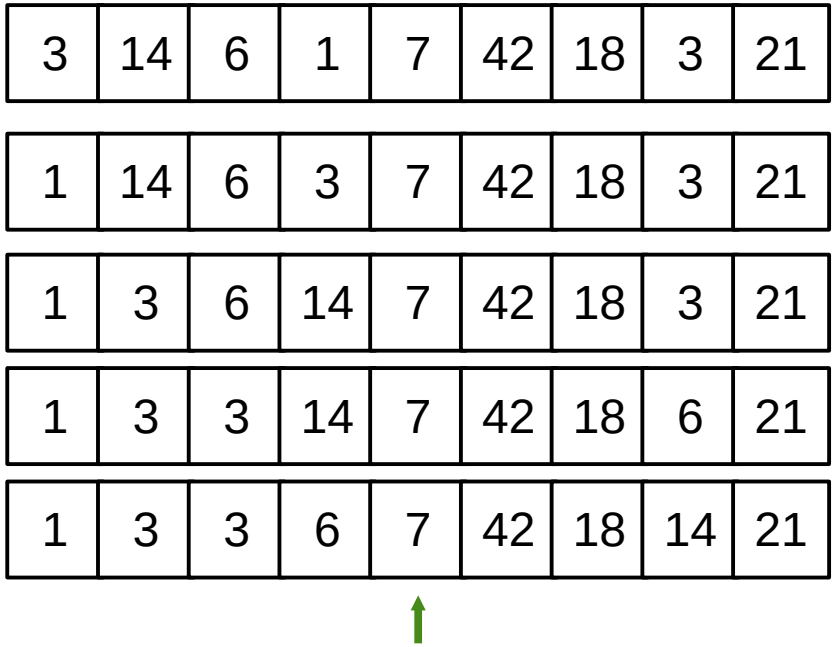
Initialize a pointer at index 0

- Left of the pointer is sorted

Find min value from pointer

- Swap with pointer value
- Increment pointer

Continue until pointer reaches end of list



# Selection Sort

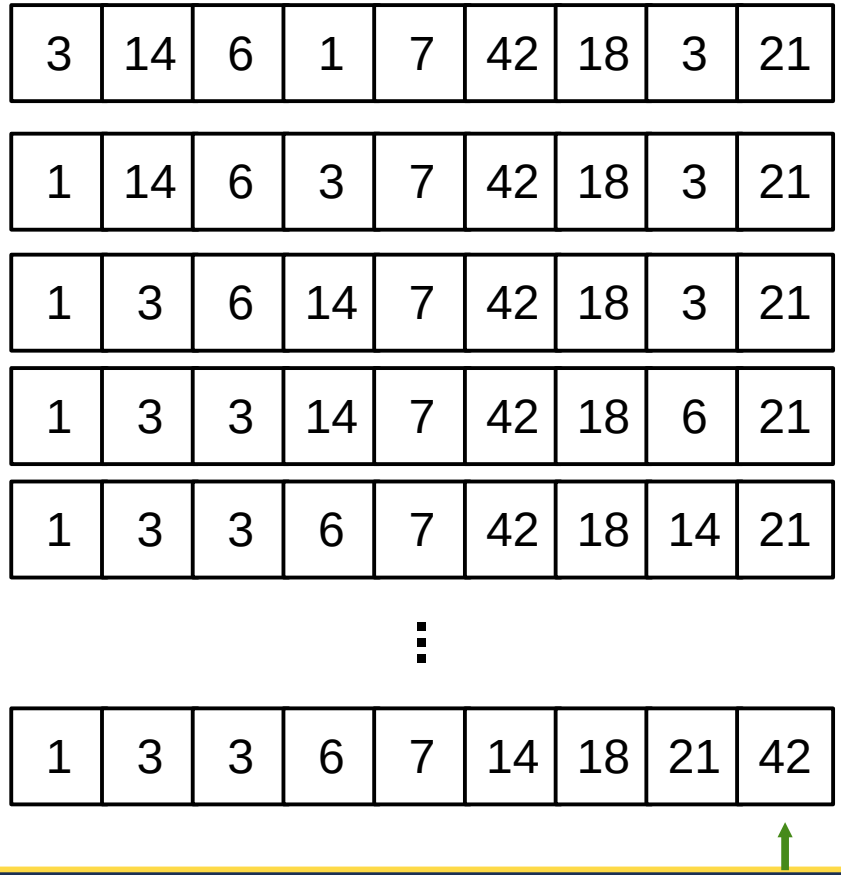
Initialize a pointer at index 0

- Left of the pointer is sorted

Find min value from pointer

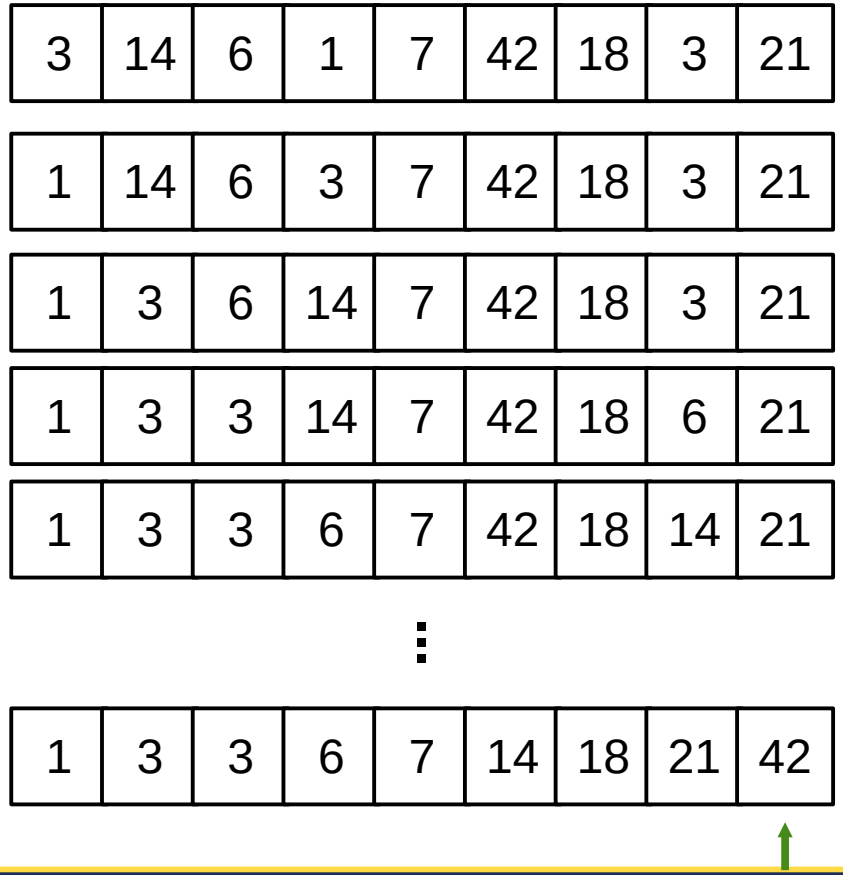
- Swap with pointer value
- Increment pointer

Continue until pointer reaches end of list



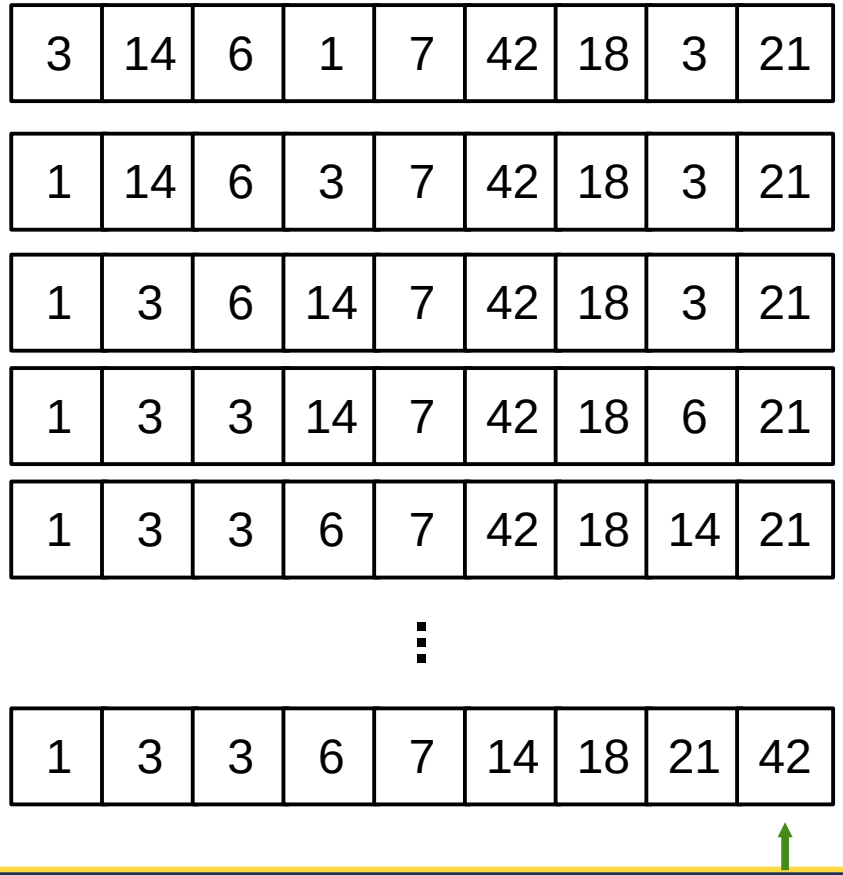
# Selection Sort

Running time?



# Selection Sort

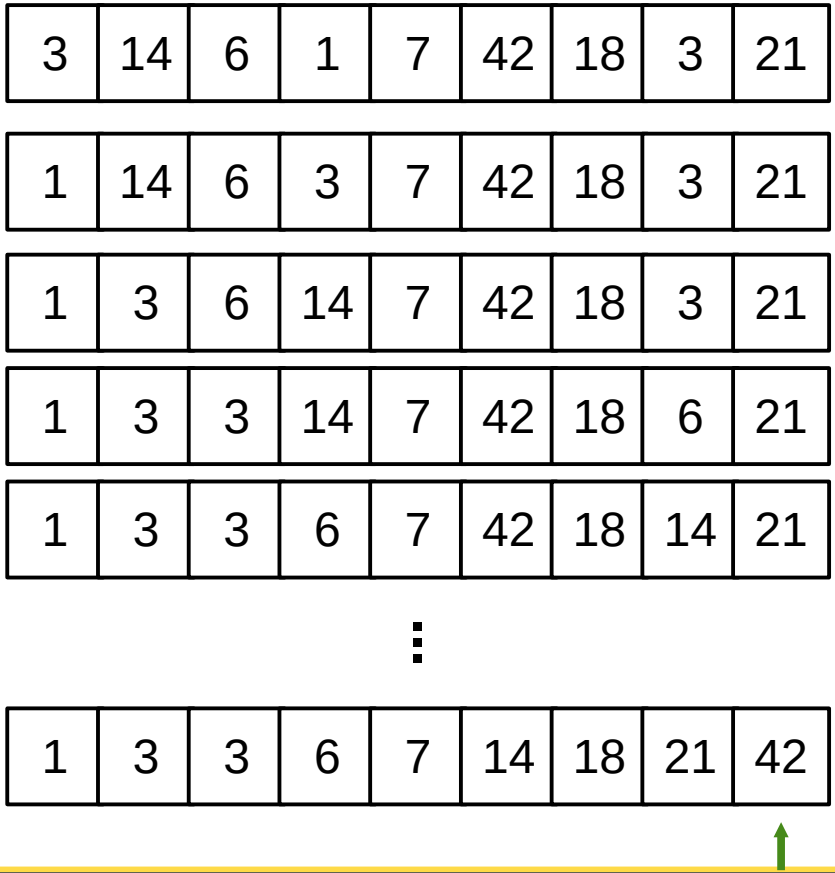
Running time? $\Theta(n^2)$



# Selection Sort

Running time? $\Theta(n^2)$

- Finding the min takes linear time
- Find min for n elements
- Then, n-1 element
- ….
- Until pointer reaches end



# Next Time

- **Lab**: searching through a directed graph!
- **Faster sorting**: merge sort